

Part 1. QCM Questions d'échauffement.

1. (2 points) Parmi ses rôles principaux, le système d'exploitation doit assurer :

- ☐ ● A l'allocation du temps processeur aux processus en cours d'exécution;
- ☐ ● B la gestion de la concurrence, ie. des accès cohérents aux données partagées par plusieurs processus;
- ☐ C l'édition des programmes et leur compilation.

2. (2 points) Un processus :

- ☐ A est statique et le programme associé représente sa dynamique d'exécution;
- ☐ ● B représente l'image de l'état du processeur et de la mémoire au cours de l'exécution d'un programme;
- ☐ C la traduction d'un langage C en assembleur.

3. (2 points) La politique d'ordonnancement d'un système d'exploitation :

- ☐ A est la manière avec laquelle les fichiers physiques sont rangés sur le disque dur;
- ☐ B est la manière avec laquelle les droits des utilisateurs sont gérés les uns par rapport aux autres;
- ☐ ● C est la manière avec laquelle le temps processeur est attribué aux différents processus.
- ☐ D est la manière avec laquelle les périphériques sont liés aux fichiers spéciaux.

4. (2 points) L'instruction C: `foo = getpid();`

- ☐ A place dans la variable `foo` l'identifiant du processus père du processus appelant;
- ☐ B place dans la variable `foo` le résultat de la création d'un processus;
- ☐ C place dans la variable `foo` l'adresse de la zone mémoire associé au processus appelant;

- ● **D** place dans la variable foo l'identifiant du processus appelant;
- ○ n'a pas d'impact sur le temps d'exécution du programme

5. (2 points) Lors de la création d'un processus par un appel à fork():

- ● **A** la terminaison du processus père n'entraîne pas la terminaison du processus fils;
- ○ B les processus père et fils ne peuvent pas être synchronisés;
- ○ les processus père et fils peuvent être synchronisés en utilisant la fonction wait() qui prend un seul argument;
- ● **D** des processus père et fils peuvent être synchronisés en utilisant la fonction waitpid() qui ne prend pas d'argument.

6. (2 points) Dans le cas d'un processus léger (thread):

- ● **A** l'accès à l'ensemble des variables du processus père est possible;
- ○ B l'accès à l'ensemble des variables du processus père n'est pas possible.

7. (2 points) Un fichier spécial:

- ● **A** est en général lié à un périphérique;
- ○ B peut être édité sans risque par un utilisateur;
- ○ C ne permet jamais un accès direct;
- ● **D** permet, en fonction de sa nature, un accès direct.

8. (2 points) perror():

- ○ A ne sert à rien;
- ● **B** permet de traduire une erreur répertoriée en un message compréhensible par un humain;
- ○ est un outil de débogage au même titre que valgrind;
- ○ ne correspond à aucune des trois affirmations précédentes.

9. (2 points) La lecture dans un tube anonyme:

- ● **A** se fait de manière directe avec read();

- ○ se fait de manière directe avec `fopen()`;
- ○ se fait de manière séquentielle avec `open()`.

10.(2 points) La redirection de sortie:

- **A** consiste à rediriger la sortie standard vers la sortie d'un tube;
- consiste à rediriger l'entrée standard vers la sortie d'un tube;
- consiste à rediriger la sortie standard vers l'entrée d'un tube;
- consiste à rediriger l'entrée standard vers l'entrée d'un tube;
- être réalisée dans le shell grâce à l'opérateur `>`.

11.(2 points) L'accès aux attributs associés à un fichier:

- ○ A peut se faire avec la fonction `stat()` qui retourne un entier correspondant à la taille du fichier;
- ● **B** peut se faire en utilisant la fonction `stat()` qui modifie une structure;
- ○ est a priori utile si on souhaite recoder la fonction `ls`.

12. Qu'est-ce qu'un processus zombie? Pourquoi a-t-on eu besoin d'introduire un tel "état"?

Un processus zombie est un processus terminé dont l'entrée dans la table des processus du système d'exploitation n'a pas encore été supprimée. Cela signifie que le processus a fini son exécution, mais son identifiant de processus (PID) et certaines informations restent dans la table des processus jusqu'à ce que le processus parent récupère son état avec un appel système comme `wait()`.

L'état de "zombie" a été introduit pour permettre au processus parent de récupérer les informations d'état (code de retour, signal d'arrêt, etc.) du processus enfant, même après la fin de son exécution. Cela permet au système de savoir si le processus s'est terminé avec succès ou s'il y a eu une erreur.

13. Que risque-t-on lorsque le nombre de zombies devient grand?

Si le nombre de processus zombies devient trop important, cela peut épuiser l'espace dans la table des processus du système d'exploitation. Chaque processus, y compris les zombies, occupe une entrée dans cette table, et une table surchargée peut empêcher la création de nouveaux processus. Cela peut rendre le système instable ou entraîner un dysfonctionnement en raison du manque de ressources pour les nouveaux processus.

14. Donnez un petit exemple de programme C provoquant cette situation à coup sûr.

Voici un exemple de programme C qui crée un processus zombie :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();

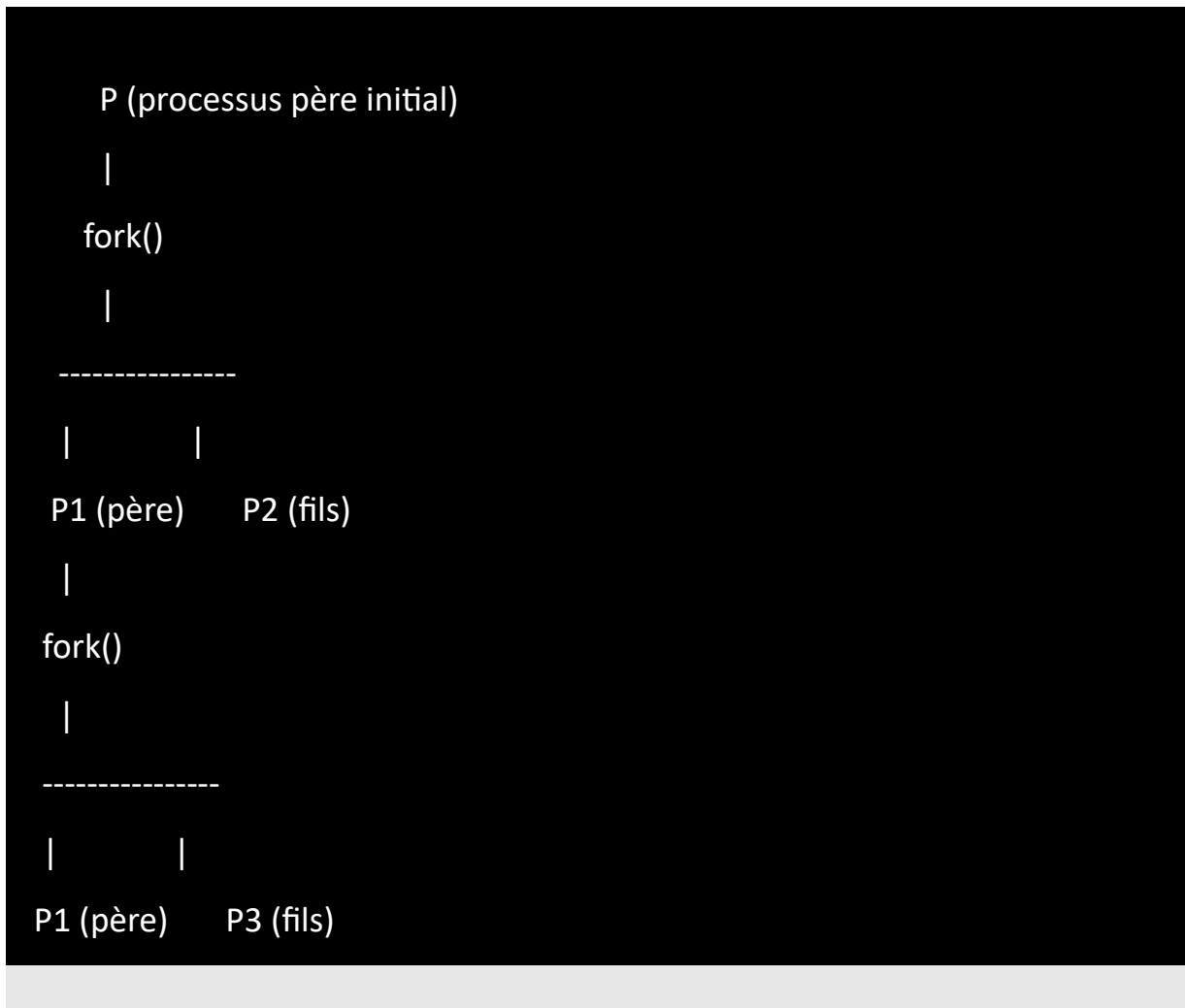
    if (pid < 0) {
        // Échec du fork
        perror("fork");
        exit(1);
    } else if (pid > 0) {
        // Processus parent : attendre pour simuler un zombie
        printf("Parent process, PID: %d\n", getpid());
        sleep(30); // Le processus parent dort, pendant ce temps l'enfant devient
        zombie
    } else {
        // Processus enfant : se termine immédiatement
        printf("Child process, PID: %d\n", getpid());
        exit(0);
    }
    return 0;
}
```

Dans cet exemple, le processus enfant se termine immédiatement en appelant `exit(0)`, tandis que le processus parent continue de s'exécuter (il dort pendant 30 secondes). Pendant ce temps, l'enfant devient un processus zombie jusqu'à ce que le parent appelle explicitement `wait()` pour récupérer son statut.

(a) Complétion de l'arbre des processus :

Un appel à la fonction `fork()` génère un nouveau processus (le processus fils) qui est une copie du processus père. Chaque appel à `fork()` divise le flux d'exécution en deux : un pour le processus père et un pour le processus fils. Le processus père continue l'exécution du programme dans une branche, tandis que le processus fils fait de même dans une autre.

L'arbre des processus peut être représenté comme suit :



Chaque `fork()` conduit à une nouvelle séparation entre les processus.

(b) Nombre total de processus créés :

Le programme commence avec un processus (le processus père). À chaque fois qu'un `fork()` est appelé, un nouveau processus est créé. Si un `fork()` est exécuté

deux fois, il créera deux nouveaux processus fils à partir des processus existants.

Nombre total de processus créés :

1. Initialement, il y a 1 processus père.
2. Le premier `fork()` crée 1 nouveau processus (1 père + 1 fils = 2 processus).
3. Le second `fork()` est exécuté par le père et le fils, ce qui double le nombre de processus (2 processus existants créent chacun un nouveau processus).

Ainsi, le nombre total de processus créés, y compris le processus père initial, est de **4 processus** (1 père + 3 nouveaux processus).

(c) Comportement de la boucle `for` :

Si une boucle `for` est exécutée dans un programme avec des appels à `fork()`, chaque processus crée une nouvelle copie de la boucle lorsqu'il est forké. Cependant, chaque processus forké exécute son propre code indépendamment. Si, par exemple, la boucle `for` contient des appels à `fork()`, chaque itération de la boucle pourrait engendrer de nouveaux processus à chaque fois.

Dans ce programme, il n'y a pas de boucle `for`, mais s'il y en avait une autour du `fork()`, elle multiplierait le nombre de processus créés. Le nombre total de processus après l'exécution dépendrait du nombre d'itérations de la boucle et du nombre d'appels à `fork()` dans la boucle.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int ticks = 0;
void handler(int signum) {
    ticks++;
    printf("tick tock: %d\n", ticks);
```



```

    if (ticks < 10) {
        alarm(1);
        // raise(9); //<-- MARK 1
    }
}

int main() {
    signal(SIGALRM, handler); //<-- MARK 2
    alarm(1); //<-- MARK 3

    while (1) {
        pause(); //<-- MARK 8
    }

    return 0;
}

```

(a) À MARK 2, quel est le but visé par l'appel à `signal()` concernant les prochaines livraisons de `SIGALRM` par le système d'exploitation?

À MARK 2, l'appel à `signal(SIGALRM, handler)` configure le gestionnaire de signal pour le signal `SIGALRM`. Cela signifie que chaque fois qu'un signal `SIGALRM` est reçu, la fonction `handler` sera appelée pour le traiter. En d'autres termes, cet appel lie le signal `SIGALRM` à la fonction `handler`, de sorte que `handler` sera exécuté chaque fois qu'un événement de signal `SIGALRM` se produit.

(b) Quel est le résultat de l'appel système à MARK 1? Quel est le nom du signal levé?

À MARK 1, l'appel `raise(9)` tente d'envoyer un signal au processus en cours. Le nombre 9 correspond au signal `SIGKILL`. Cependant, l'appel à `raise()` avec un signal `SIGKILL` est incorrect ici, car `SIGKILL` ne peut pas être capturé, ignoré ou bloqué, et il ne peut pas être utilisé avec `raise()` dans ce contexte.

En réalité, l'utilisation de `raise(SIGKILL)` provoquerait une erreur de compilation car le signal `SIGKILL` est défini comme une constante et n'est pas censé être envoyé par `raise()`. En conséquence, si le programme était compilé avec `raise(SIGKILL)`, il produirait une erreur de compilation, ou il pourrait éventuellement être traité comme une tentative d'envoi d'un signal invalide selon l'implémentation du compilateur et de la bibliothèque de temps d'exécution.

En résumé, le signal levé est `SIGKILL`, mais l'appel est incorrect et entraînera une erreur.

(c) Combien de signaux SIGALRM sont livrés par le programme? Expliquez.

Le programme livrera un total de **10 signaux SIGALRM**.

Explication :

1. Lors du démarrage du programme, l'appel `alarm(1)` dans la fonction `main()` planifie la livraison d'un signal `SIGALRM` après 1 seconde.
2. Lorsque le signal `SIGALRM` est livré, la fonction handler est appelée.
3. Dans la fonction handler, chaque appel à `alarm(1)` planifie un nouveau signal `SIGALRM` dans 1 seconde.
4. La variable `ticks` est incrémentée et imprimée à chaque fois que le gestionnaire est appelé.
5. Le gestionnaire arrête de planifier de nouveaux signaux lorsque `ticks` atteint 10. À ce point, `ticks` est égal à 10, et le `if (ticks < 10)` est faux, donc `alarm(1)` n'est pas appelé à nouveau.

Ainsi, le signal `SIGALRM` est planifié 10 fois (une fois par seconde) jusqu'à ce que `ticks` atteigne 10. Chaque planification correspond à un signal `SIGALRM` livré.

(d) Quel est le rôle de pause() à MARK 8? Est-ce alors mieux ou pire de le remplacer avec le code suivant sans pause() ? while(1);

Rôle de pause() à MARK 8 :

La fonction pause() met le processus en attente jusqu'à ce qu'un signal arrive. Lorsqu'un signal arrive, le processus reprend l'exécution et le signal est traité. Dans le contexte du programme donné, pause() permet au programme de rester inactif et d'attendre les signaux SIGALRM qui sont livrés chaque seconde. Ainsi, pause() est utilisé pour suspendre l'exécution du processus et ne pas utiliser inutilement le processeur pendant ce temps d'attente.

Remplacement de pause() par while(1); :

Si vous remplacez pause() par while(1);, le programme entrera dans une boucle infinie et occupera le processeur de manière active sans dormir. Cela consomme inutilement des ressources processeur, car le programme tournera sans fin, même si ce n'est que pour vérifier la condition de la boucle.

Comparaison :

- **Avec pause():** Le programme attend de manière passive la livraison des signaux, ce qui est plus efficace en termes d'utilisation du processeur.
- **Avec while(1);:** Le programme utilise de manière active le processeur pour effectuer une tâche qui ne fait rien d'utile. Cela est généralement moins efficace et peut conduire à une consommation élevée de ressources CPU.

Conclusion : Utiliser pause() est mieux que while(1); dans ce contexte, car pause() est conçu pour suspendre l'exécution du programme sans consommer activement des ressources processeur, ce qui est plus approprié pour un programme qui attend des signaux.

Pour modifier le programme trier afin qu'il prenne en compte une sous-partie du fichier, vous devez ajouter deux arguments supplémentaires pour spécifier la position de départ et le nombre d'entiers à traiter. Voici comment vous pouvez ajuster les lignes 14 et 15 pour prendre en compte ces nouveaux paramètres.

Code Modifié

Voici le code mis à jour avec les modifications nécessaires :

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <fcntl.h>

#include <sys/stat.h>

#include <sys/types.h>


// Fonction externe pour trier un tableau d'entiers
extern void sort(int *array, unsigned nb_integers);


int main(int argc, char *argv[]) {
    int fd;

    unsigned start_pos, num_integers;

    struct stat file_attributes;

    int *buf;

    off_t len;


    // Vérification des arguments
    if (argc != 4) {
        fprintf(stderr, "Usage: %s <fichier> <start_pos> <num_integers>\n",
argv[0]);
```

```
    exit(EXIT_FAILURE);
}

// Lecture des arguments de la ligne de commande
start_pos = atoi(argv[2]);
num_integers = atoi(argv[3]);

// Ouverture du fichier en lecture seule
fd = open(argv[1], O_RDONLY);
if (fd < 0) {
    perror("open");
    exit(EXIT_FAILURE);
}

// Obtention des attributs du fichier
if (fstat(fd, &file_attributes) < 0) {
    perror("fstat");
    exit(EXIT_FAILURE);
}

// Calcul de la taille des données à lire
len = num_integers * sizeof(int);

// Allocation de la mémoire pour le buffer
buf = malloc(len);
if (buf == NULL) {
```

```
perror("malloc");
exit(EXIT_FAILURE);
}

// Positionnement dans le fichier à la position de départ
if (lseek(fd, start_pos * sizeof(int), SEEK_SET) < 0) {
    perror("lseek");
    exit(EXIT_FAILURE);
}

// Lecture des entiers à partir de la position de départ
if (read(fd, buf, len) < 0) {
    perror("read");
    exit(EXIT_FAILURE);
}

// Tri des entiers
sort(buf, num_integers);

// Écriture des résultats sur la sortie standard
if (write(STDOUT_FILENO, buf, len) < 0) {
    perror("write");
    exit(EXIT_FAILURE);
}

// Libération de la mémoire et fermeture du fichier
```

```
free(buf);  
close(fd);  
  
return 0;  
}
```

Explication des Modifications

1. Lecture des arguments :

- Les arguments supplémentaires sont lus avec `atoi()`, qui convertit les chaînes de caractères en entiers.
- `start_pos` représente la position du premier entier à traiter, et `num_integers` est le nombre d'entiers à lire et trier.

2. Positionnement dans le fichier :

- `lseek(fd, start_pos * sizeof(int), SEEK_SET)` déplace le curseur de lecture dans le fichier à la position désirée. Notez que nous multiplions `start_pos` par la taille d'un entier pour obtenir la bonne position en octets.

3. Lecture et traitement :

- La lecture du fichier est maintenant limitée au segment spécifié (`num_integers * sizeof(int)` octets).
- Le tableau est trié en utilisant la fonction `sort`.

4. Gestion des erreurs et nettoyage :

- Le code vérifie les erreurs pour les opérations de fichier et de mémoire, et libère les ressources allouées avant de terminer.

Avec ces modifications, le programme trier peut traiter une sous-partie du fichier spécifiée par les nouveaux arguments.

Voici un squelette de programme en C qui crée deux processus fils, chacun connecté à un tube distinct au processus père. Le processus père ne peut lire

que depuis les tubes, et les processus fils ne peuvent écrire que dans leurs tubes respectifs. Toutes les extrémités de tubes non utiles sont fermées.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int pipe1[2], pipe2[2]; // Descripteurs pour les deux tubes
    pid_t pid1, pid2;

    // Création du premier tube
    if (pipe(pipe1) == -1) {
        perror("pipe1");
        exit(EXIT_FAILURE);
    }

    // Création du second tube
    if (pipe(pipe2) == -1) {
        perror("pipe2");
        exit(EXIT_FAILURE);
    }

    // Création du premier processus fils
    pid1 = fork();
    if (pid1 < 0) {
        perror("fork1");
    }
}
```



```
    exit(EXIT_FAILURE);
}

if (pid1 == 0) {
    // Processus fils 1
    close(pipe1[0]); // Ferme l'extrémité de lecture du tube1
    close(pipe2[0]); // Ferme les extrémités non utilisées
    close(pipe2[1]);

    // Écriture dans le tube1 (écriture seulement pour le fils)
    // ...

    close(pipe1[1]); // Ferme l'extrémité d'écriture après utilisation
    exit(EXIT_SUCCESS);
}

// Création du second processus fils
pid2 = fork();
if (pid2 < 0) {
    perror("fork2");
    exit(EXIT_FAILURE);
}

if (pid2 == 0) {
    // Processus fils 2
    close(pipe2[0]); // Ferme l'extrémité de lecture du tube2
```

```
close(pipe1[0]); // Ferme les extrémités non utilisées
close(pipe1[1]);

// Écriture dans le tube2 (écriture seulement pour le fils)
// ...

close(pipe2[1]); // Ferme l'extrémité d'écriture après utilisation
exit(EXIT_SUCCESS);
}

// Processus père
close(pipe1[1]); // Ferme l'extrémité d'écriture du tube1
close(pipe2[1]); // Ferme l'extrémité d'écriture du tube2

// Lecture des tubes (lecture seulement pour le père)
// ...

close(pipe1[0]); // Ferme l'extrémité de lecture après utilisation
close(pipe2[0]);

// Attente de la fin des processus fils
wait(NULL);
wait(NULL);

return 0;
}
```

Explication

1. Création des tubes :

- pipe(pipe1) et pipe(pipe2) créent deux tubes, chacun avec une extrémité de lecture (index 0) et une extrémité d'écriture (index 1).

2. Création des processus fils :

- fork() est appelé pour créer le premier et le second processus fils.

3. Configuration des tubes pour les processus fils :

- Dans chaque processus fils, les extrémités non utilisées des tubes sont fermées. Chaque fils garde seulement l'extrémité d'écriture du tube qui lui est attribué.
- Après avoir effectué l'écriture (commentée ici), chaque fils ferme l'extrémité d'écriture et termine.

4. Configuration des tubes pour le processus père :

- Le processus père ferme les extrémités d'écriture des tubes, car il ne les utilise pas. Il garde seulement les extrémités de lecture.
- Après avoir effectué la lecture (commentée ici), le père ferme les extrémités de lecture et attend la fin des processus fils.

5. Gestion des erreurs :

- Les erreurs de pipe() et fork() sont traitées pour garantir que le programme se termine proprement en cas d'échec.

Ce squelette configure les tubes et les processus conformément aux exigences spécifiées. Vous pouvez ajouter le code nécessaire pour lire et écrire dans les sections commentées selon vos besoins.